

KHOA CÈNG NGHỆ THỀNG TIN



Công nghệ phần mềm

Chương IV: Thiết kế phần mềm



Nội dung

- ➔ • Tổng quan về thiết kế phần mềm
 - Thiết kế phần mềm trong kỹ nghệ phần mềm
 - Nguyên lý thiết kế phần mềm
 - Tiến trình thiết kế phần mềm
 - Các khái niệm thiết kế cơ sở
 - Các phương pháp thiết kế
 - Hướng cấu trúc
 - Hướng đối tượng
 - Hướng khía cạnh

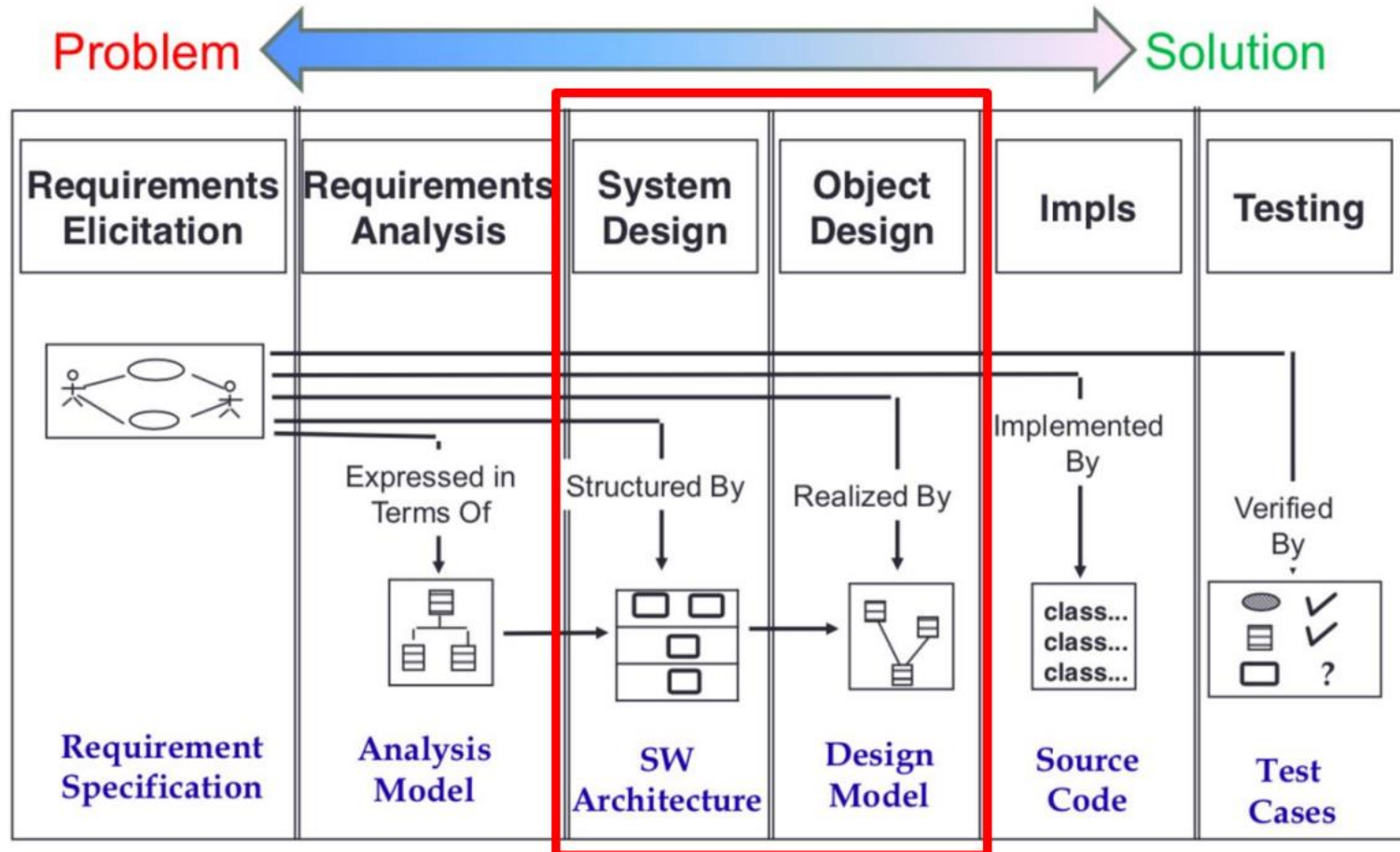


Nội dung

- Thiết kế hướng đối tượng:
 - Cách tiếp cận hướng đối tượng
 - Đặc trưng của thiết kế hướng đối tượng
 - Các ưu nhược điểm của thiết kế hướng đối tượng
 - Mẫu thiết kế - Design patterns
 - Một số nhiệm vụ thiết kế đề cập trong bài học:
 - Thiết kế dữ liệu
 - Thiết kế xử lý
 - Thiết kế giao diện



Tổng quan về thiết kế phần mềm





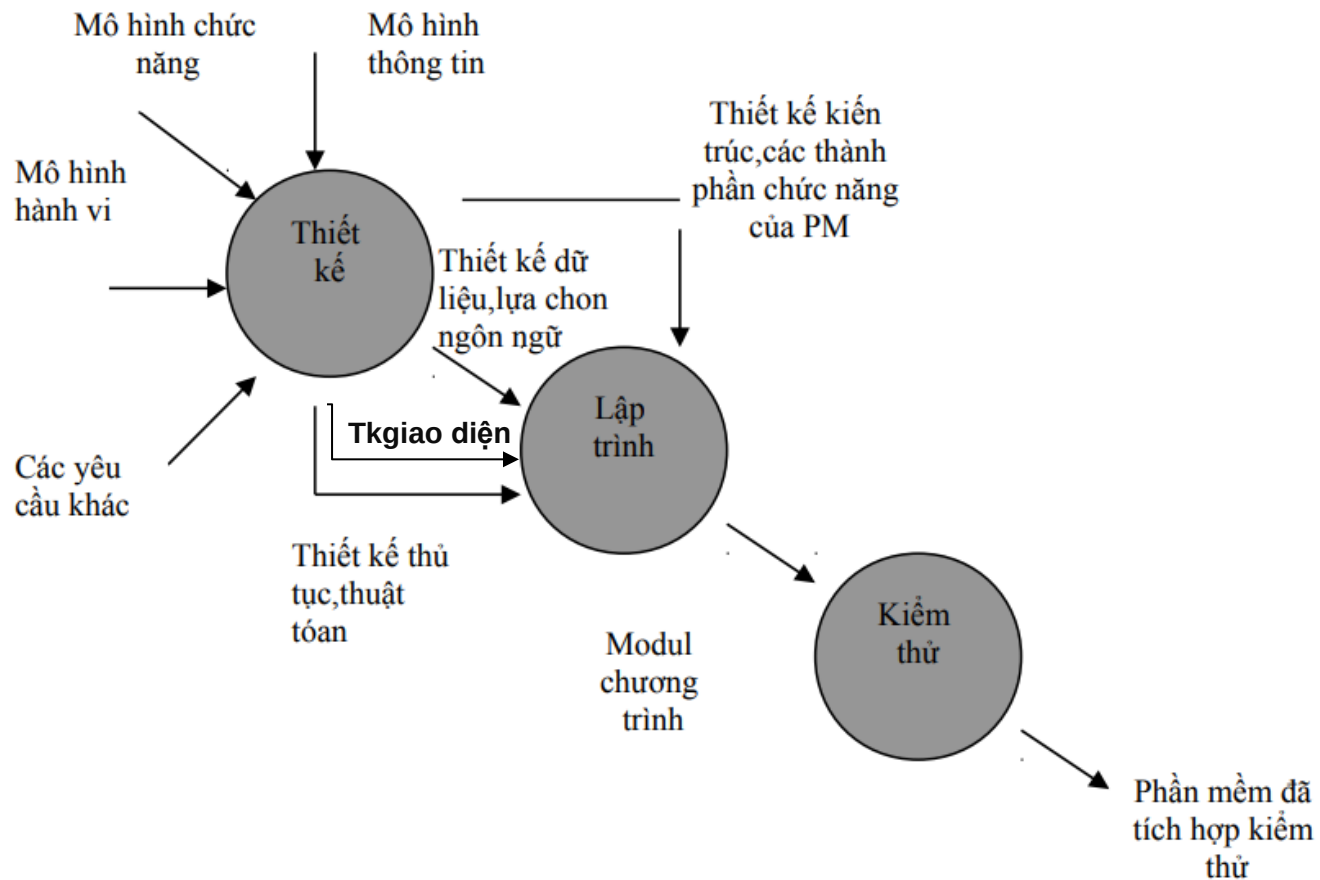
Tổng quan về thiết kế phần mềm

- Thiết kế là chuyển bản đặc tả yêu cầu thành mô tả thiết kế đủ chi tiết để người lập trình có thể chuyển thành chương trình với 1 ngôn ngữ lập trình, vận hành đáp ứng được yêu cầu đặt ra.
- Bao gồm:
 - Thiết kế hệ thống: Thiết kế cấu trúc ngoài của phần mềm.
 - Thiết kế chương trình: Thiết kế chi tiết cấu trúc bên trong của phần mềm. Thiết kế tính năng từng môđun và giao diện tương ứng



Tổng quan về thiết kế phần mềm

- Thiết kế phần mềm trong kỹ nghệ PM





Tổng quan về thiết kế phần mềm

- Vai trò của thiết kế:
 - Tạo tài liệu cho pha cài đặt phần mềm.
 - Là công cụ giao tiếp giữa các những người tham gia phát triển, cơ sở đảm bảo chất lượng hệ thống.
 - Cung cấp đầy đủ thông tin cho những người bảo trì hệ thống trong việc bảo trì phần mềm sau này.



Tổng quan về thiết kế phần mềm

- Cấu trúc thiết kế
 - Phần mềm là tập các mô đun tương tác lẫn nhau.
 - **Mô đun hóa** là chìa khóa cho phần mềm tốt
 - Mục tiêu thiết kế là xác định:
 - Các mô đun chức năng
 - Cách thức cài đặt mô đun
 - Tương tác giữa các mô đun



Nội dung

- Tổng quan về thiết kế phần mềm
 - Thiết kế phần mềm trong kỹ nghệ phần mềm
 - – Nguyên lý thiết kế phần mềm
 - Tiến trình thiết kế phần mềm
 - Các khái niệm thiết kế cơ sở
 - Các phương pháp thiết kế
 - Hướng cấu trúc
 - Hướng đối tượng
 - Hướng khía cạnh



Nguyên lý thiết kế phần mềm

1. Không bị bó buộc vào một cách nhìn hạn chế nào. Nó cần được lựa chọn từ các giải pháp có thể.
2. Cho phép lần ngược lại mô hình phân tích.
 - Các mô đun & các yêu cầu không nhất thiết phải tương ứng 1-1
 - Nhưng phải kiểm tra được sự thỏa mãn các yêu cầu



Nguyên lý thiết kế phần mềm

3. Không nên tạo lại các thiết kế (giải pháp) đã có, mà cần tái sử dụng tối đa chúng.
4. Mô hình thiết kế (giải pháp) nên tiến gần đến mô hình thế giới thực (bài toán).
5. Biểu diễn thiết kế phải **nhất quán** và có **tính tích hợp**:
 - ✓ thiết kế do nhiều người tiến hành song song
 - ✓ phải thống nhất cách biểu diễn, thống nhất giao diện



Nguyên lý thiết kế phần mềm

6. Thiết kế **cần có cấu trúc** để dễ hiểu, dễ thay đổi: Phải được modun hóa, phân cấp.
7. Thiết kế không phải là mã hóa.
8. Thiết kế **cần được đánh giá chất lượng** ngay trong khi được tạo ra (tính kết dính, tính ghép nối, hiệu quả thuật toán).
9. Thiết kế **cần được thẩm định** để tránh các lỗi mang tính hệ thống.



Nội dung

- Tổng quan về thiết kế phần mềm
 - Thiết kế phần mềm trong kỹ nghệ phần mềm
 - Nguyên lý thiết kế phần mềm
 - – Tiến trình thiết kế phần mềm
 - Các khái niệm thiết kế cơ sở
 - Các phương pháp thiết kế
 - Hướng cấu trúc
 - Hướng đối tượng
 - Hướng khía cạnh



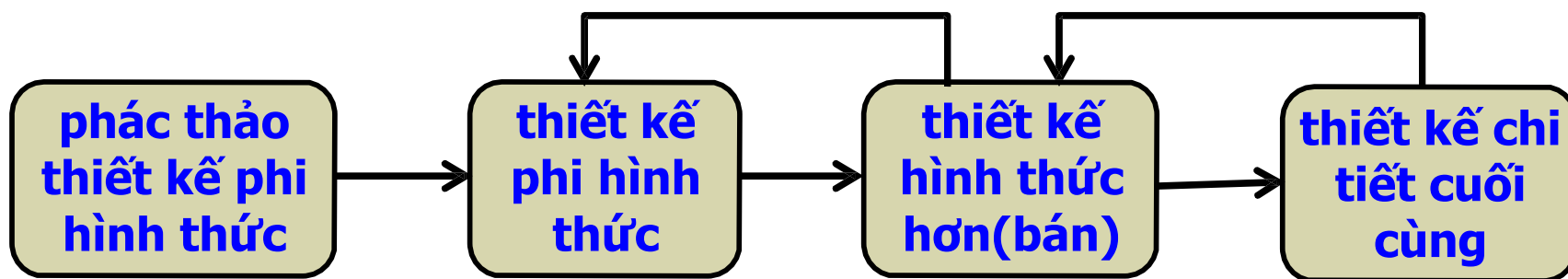
Tiến trình thiết kế phần mềm

- Các giai đoạn trong thiết kế phần mềm:
Thiết kế phần mềm trải qua ba giai đoạn
 - Nghiên cứu và tìm hiểu các kết quả phân tích và đặc tả yêu cầu phần mềm ở công đoạn trước.
 - Lựa chọn các giải pháp thiết kế. Chọn giải pháp dựa vào kinh nghiệm của người thiết kế.
 - Triển khai công việc thiết kế.



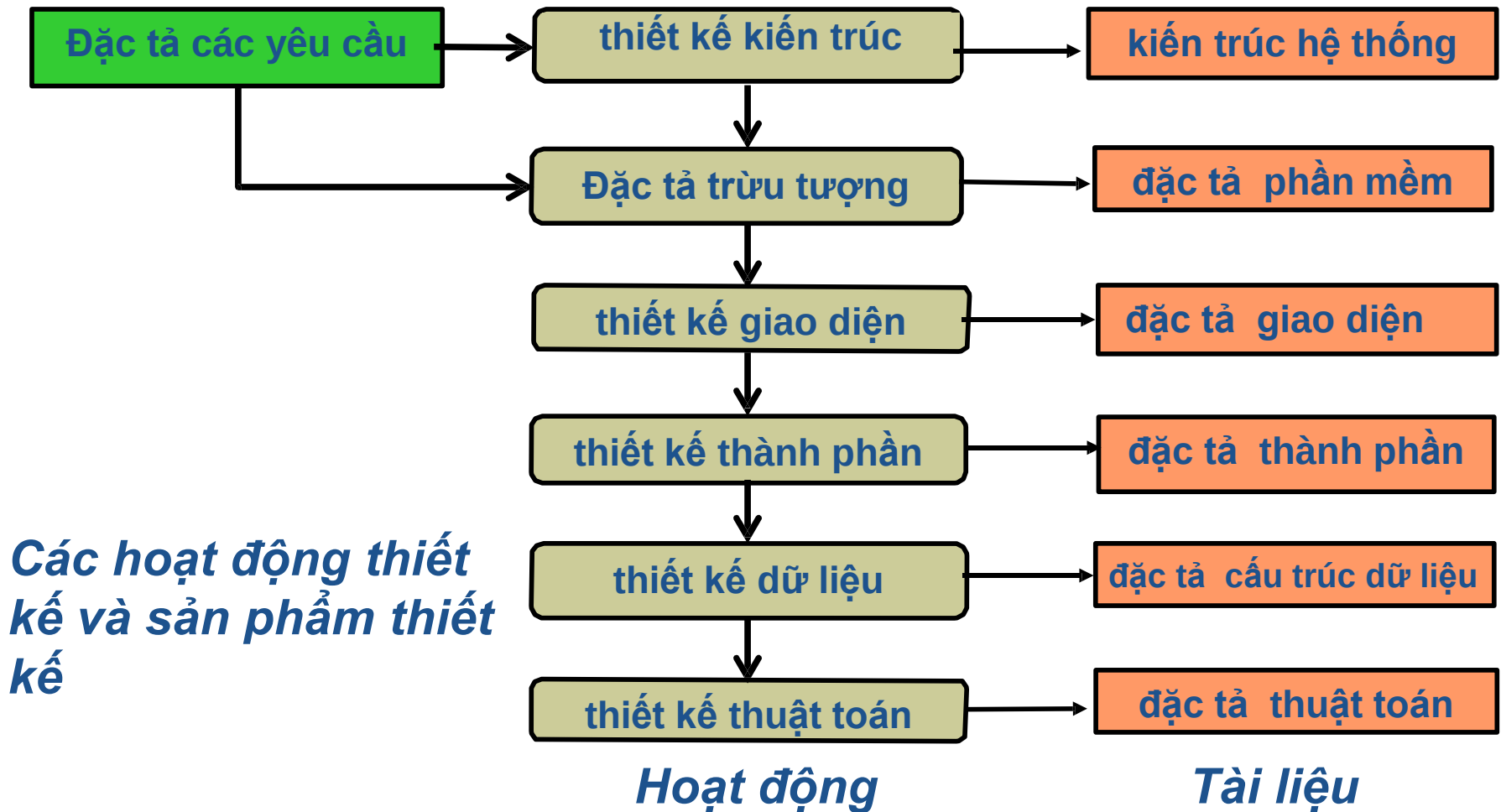
Tiến trình thiết kế phần mềm

- Quá trình thiết kế: Là quá trình tăng cường hình thức hóa và luôn quay lại các thiết kế đúng đắn và ít hình thức hóa hơn trước đó để kiểm tra và hoàn chỉnh.





Tiến trình thiết kế phần mềm





Tiến trình thiết kế phần mềm

- Quá trình thiết kế: Bao gồm các hoạt động
 - Thiết kế kiến trúc:
 - Là giai đoạn đầu của quá trình thiết kế hệ thống
 - Là quá trình xác định:
 - Những hệ thống con hoặc mô đun
 - Và một khung làm việc giúp kiểm soát và giao tiếp giữa các hệ thống con/mô đun này.
 - Đặc tả trừu tượng:
 - Đối với mỗi hệ con, một đặc tả trừu tượng
 - Các dịch vụ mà nó cung cấp
 - Và các ràng buộc tuân theo phải cung cấp



Tiến trình thiết kế phần mềm

- Quá trình thiết kế (tiếp):
 - Thiết kế giao diện:
 - Là thiết kế giao diện của từng hệ con và với các hệ con khác.
 - Thiết kế thành phần:
 - Các dịch vụ được cung cấp bởi hệ con được phân chia qua các thành phần hợp thành của hệ con đó
 - Thiết kế dữ liệu:
 - Thiết kế CSDL được dùng trong việc thực hiện hệ thống ở các mức chi tiết đặc tả khác nhau.
 - Thiết kế thuật toán:
 - Các thuật toán được thiết kế chi tiết và đặc tả



Tiến trình thiết kế phần mềm

- Trong quá trình thiết kế, các hoạt động này được lặp lại cho các hệ con cho đến khi rõ ràng, đủ chi tiết để có thể minh định.
- Phương pháp tiếp cận thường dùng trong thiết kế là tiếp cận từ trên xuống (top-down)
- Các bước tiến hành thiết kế theo quan điểm quản lý dự án:
 1. Thiết kế sơ bộ
 2. Thiết kế chi tiết



Nội dung

- Tổng quan về thiết kế phần mềm
 - Thiết kế phần mềm trong kỹ nghệ phần mềm
 - Nguyên lý thiết kế phần mềm
 - Tiến trình thiết kế phần mềm
 - – Các khái niệm thiết kế cơ sở
 - Các phương pháp thiết kế
 - Hướng cấu trúc
 - Hướng đối tượng
 - Hướng khía cạnh



Các khái niệm thiết kế cơ sở

- Trừu tượng hóa: Trừu tượng hóa dữ liệu, thủ tục, điều khiển.
- Làm mịn: Chi tiết hóa các trừu tượng theo ý đồ.
- Tính môđun: Phân chia dữ liệu và chức năng
- Kiến trúc: Cấu trúc tổng thể của phần mềm
- Thủ tục: Thuật toán để thực hiện chức năng
- Che dấu thông tin: Điều khiển bằng giao diện



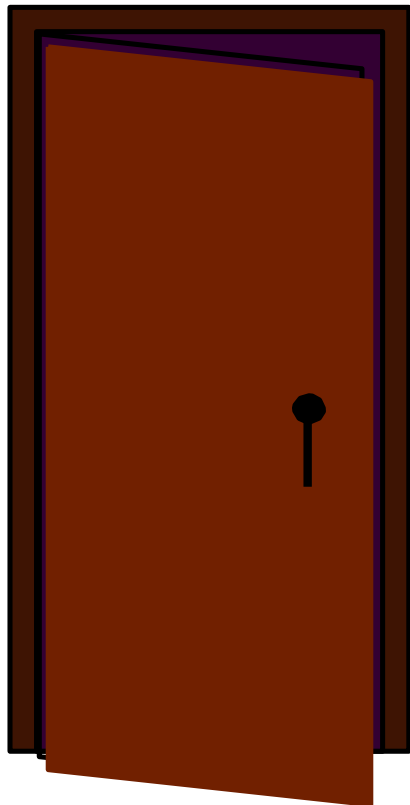
Trừu tượng hóa

- Là quá trình ánh xạ một sự vật/hiện tượng của thế giới thực thành một khái niệm logic.
- Giảm thiểu tinh lọc các chi tiết nhằm tập trung vào một số khái niệm/vấn đề quan tâm tại một thời điểm.
- Có nhiều mức trừu tượng khác nhau:
 - Cho phép con người tập trung vào giải quyết vấn đề mà không cần bận tâm đến chi tiết.
 - Biểu diễn vấn đề bằng một cấu trúc tự nhiên.



Trừu tượng hóa

1. Trừu tượng dữ liệu:



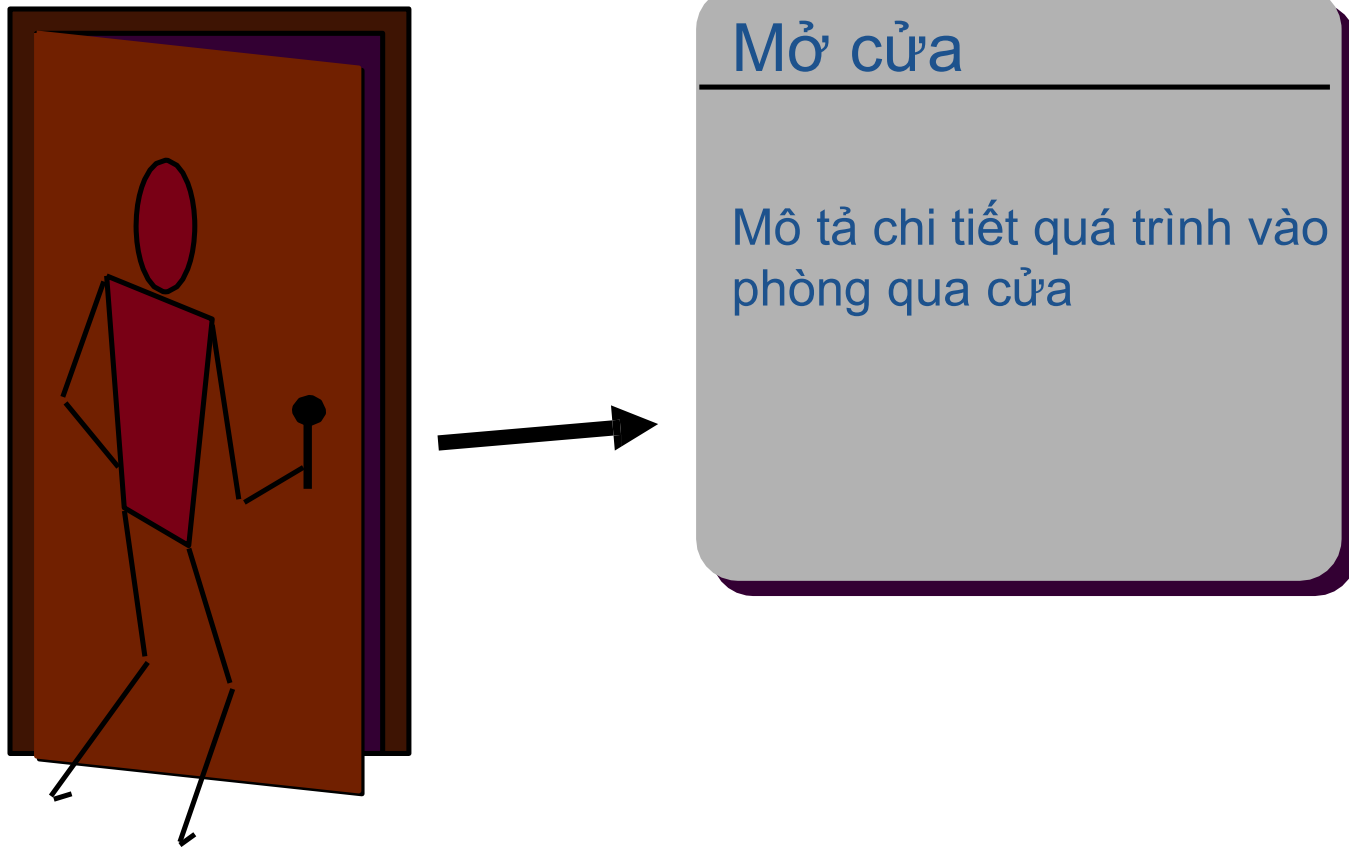
Cửa

mã số: 256AD loại: *cửa mở kéo*
hướng mở: *ra bên trái*
cao: *2.3 m*
rộng: *0.85 m*
trọng lượng: *120*
màu: *nâu cánh dán*



Trừu tượng hóa

2. Trừu tượng thủ tục :





Làm mịn từng bước

Mở cửa

Bước đến gần cửa
Đưa chìa khóa vào ổ xoay

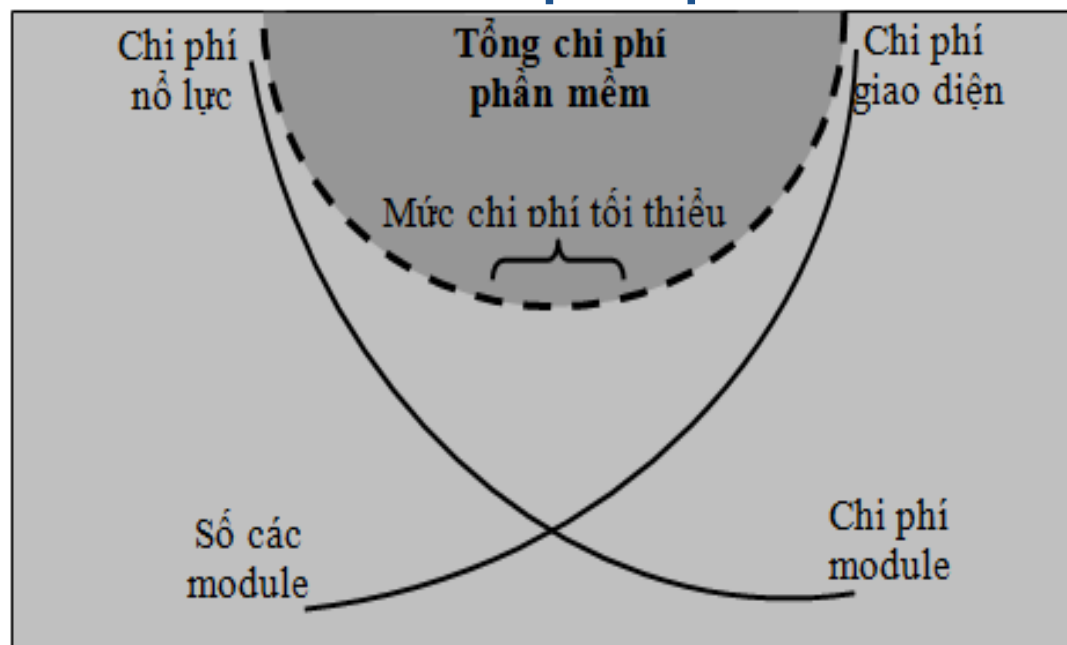
Mở cửa →
Bước qua vào phòng
Đóng cửa lại

Lặp lại cho đến khi chốt khóa bật ra
Nếu chốt không mở thì
Rút khóa ra, tìm chìa khác phù
hợp, cắm vào ổ khóa, tiếp tục
xoay cho đến khi mở được



Tính mô đun

- Phần mềm được chia thành các thành phần có tên riêng biệt và xác định được địa chỉ, gọi là các mô đun.
- Tính mô đun và chi phí phần mềm:





Che giấu thông tin

- Sử dụng môđun thông qua các giao diện, danh sách tham số và giá trị trả lại.
- Không cần biết cách thức cài đặt của nó:
 - Thuật toán
 - Cấu trúc dữ liệu
 - Giao diện ngoại lai (môđun thứ cấp, thiết bị vào ra)
 - Tài nguyên hệ thống



Che dấu thông tin

- Lý do che dấu thông tin:
 - Giảm hiệu ứng phụ khi sửa đổi mô đun
 - Giảm tác động của thiết kế tổng thể lên thiết kế cục bộ
 - Nhấn mạnh trao đổi thông tin thông qua giao diện
 - Loại bỏ việc sử dụng dữ liệu dùng chung
 - Hướng tới sự đóng gói chức năng là thuộc tính của một thiết kế tốt

➡ **Tạo ra các sản phẩm phần mềm tốt hơn!**



Nội dung

- Tổng quan về thiết kế phần mềm
 - Thiết kế phần mềm trong kỹ nghệ phần mềm
 - Nguyên lý thiết kế phần mềm
 - Tiến trình thiết kế phần mềm
 - Các khái niệm thiết kế cơ sở
 - – Một số phương pháp thiết kế
 - Hướng cấu trúc
 - Hướng đối tượng
 - Hướng khía cạnh



Các phương pháp thiết kế

- Phương pháp thiết kế:
 - Phương pháp hướng cấu trúc
 - Phương pháp hướng đối tượng
 - Phương pháp hướng khía cạnh (AOSD)
- Các phương pháp thiết kế trên thường trợ giúp một số cách nhìn nhận hệ thống:
 - Nhìn nhận cấu trúc
 - Nhìn nhận quan hệ thực thể
 - Nhìn nhận dòng dữ liệu (hướng chức năng),
 - Nhìn nhận trao đổi thông điệp (hướng ĐT)



Các phương pháp thiết kế

- Phương pháp cấu trúc:
 - Là phương pháp đầu tiên ra đời vào những đầu năm thập kỷ 1970
 - Dựa vào các chức năng của phần mềm
- Phương pháp hướng đối tượng:
 - Ra đời từ những năm thập kỷ 1990.
 - Dựa vào các thực thể (các đối tượng) trong bài toán.
 - Được xây dựng trên nền tảng:
 - Hướng cấu trúc
 - và sự trừu tượng hóa dữ liệu.



Các phương pháp thiết kế

- Hạn chế của các phương pháp:
 - Phương pháp hướng cấu trúc: Lộ rõ yếu điểm khi
 - Độ phức tạp của một chương trình tăng lên
 - Cấu trúc dữ liệu phức tạp lên
 - Sử dụng dữ liệu dùng chung là khó cho việc làm theo nhóm
 - Với hệ thống thường xuyên thay đổi vì khó thực thi những thay đổi trong nguyên tắc nghiệp vụ hay cách hoạt động của hệ thống.
 - Phương pháp hướng đối tượng:
 - Chỉ dừng lại ở mức tĩnh (Sinh – Hoạt – Tử)
 - Thiếu tính khả mở, tiến hóa



Các phương pháp thiết kế

- Hạn chế của các phương pháp trên:
 - Một hệ thống thực tế là sự đan xen giữa các yêu cầu.
 - Một yêu cầu nghiệp vụ thường được bổ sung thêm các yêu cầu khác, gây nên sự chồng chéo giữa các yêu cầu.
 - Từ đó hệ thống thiết kế theo hướng đối tượng sẽ khó bảo trì, phát triển.
 - Khi bổ sung thêm một nghiệp vụ phải dẫn đến thay đổi tất cả các yêu cầu nghiệp vụ liên quan.



Các phương pháp thiết kế

- Phương pháp hướng khía cạnh:
 - Là phương pháp sử dụng mô hình AOP (Aspect-Oriented Programming).
 - Được xây dựng dựa trên phương pháp hướng cấu trúc và hướng đối tượng. Bổ sung thêm các khái niệm và cấu trúc để module hóa các concern đan xen.
 - Khác nhau cơ bản của hướng khía cạnh là cách quản lý các nhiệm vụ đan xen (*cross-cutting concerns*) và các nhiệm vụ thành phần (*core concerns*)



Các phương pháp thiết kế

- Phương pháp hướng khía cạnh: (Tiếp)
 - Các nhiệm vụ đan xen (*cross-cutting concerns*) được hệ thống đóng gói trong các aspect.
 - Trong thiết kế quyết định các module khác nhau (gồm các core concern và các aspect) sẽ được gắn kết với nhau như thế nào để tạo nên hệ thống cuối cùng.



Các phương pháp thiết kế

- Phương pháp hướng khía cạnh: (Tiếp)

Ưu điểm:

- Tách biệt hơn các chức năng của các module độc lập
- Tính module hóa cao hơn
- Phát triển hệ thống dễ dàng hơn
- Kết nối muện thiết kế
- Tính tái sử dụng mã
- Giảm thời gian cài đặt và giá thành phần mềm



Các phương pháp thiết kế

- Phương pháp hướng khía cạnh: (Tiếp)

Một số đặc điểm gây trở ngại:

- Chưa giải quyết được vấn đề của phương pháp OOP về tính khả mở và tiến hóa. Nó chỉ đưa ra cách giải quyết bài toán tốt hơn theo hai hướng phương pháp cấu trúc và OOP.
- Không giải quyết được cho các thiết kế cầu thả.
- Phá vỡ tính đóng gói trong OOP (encapsulation)



Nội dung

- • Thiết kế hướng đối tượng:
 - Cách tiếp cận hướng đối tượng
 - Đặc trưng của thiết kế hướng đối tượng
 - Các ưu nhược điểm của thiết kế hướng đối tượng
 - Mẫu thiết kế - Design patterns
 - Một số nhiệm vụ thiết kế đề cập trong bài học:
 - Thiết kế dữ liệu
 - Thiết kế xử lý
 - Thiết kế giao diện



Thiết kế hướng đối tượng

- Các thuật ngữ trong phương pháp hướng đối tượng:
 - OOA liên quan đến việc phát triển một mô hình đối tượng của miền ứng dụng.
 - OOD liên quan đến việc mô tả tất cả các đối tượng và sự tương tác giữa chúng trong hệ thống để có thể cài đặt được các yêu cầu.
 - OOP liên quan đến việc thực hiện cài đặt một bản thiết kế bằng cách sử dụng một ngôn ngữ lập trình OO như Java hay .NET, C++ ...



Cách tiếp cận hướng đối tượng

- Là lối tiếp cận mà tư duy vấn đề:
 - Ánh xạ các thành phần bài toán vào các đối tượng ngoài đời thực.
 - Chia ứng dụng thành các thành phần nhỏ gọi là các đối tượng tương đối độc lập với nhau.
 - Mỗi đối tượng là các kết hợp giữa code và dữ liệu
 - Là một đơn vị duy nhất, nó có tên riêng và tham chiếu đến nó qua tên.
 - Có khả năng nhận vào các thông điệp, xử lý dữ liệu (bên trong của nó), và gửi ra hay trả lời đến các đối tượng khác hay đến môi trường.



Cách tiếp cận hướng đối tượng

- Giống cách giải quyết vấn đề của con người. Bao gồm việc nhận diện các object và chỉ ra cách sử dụng chúng theo đúng thứ tự nhằm giải quyết bài toán thực tế. Có thể chia làm 4 bước:
 - Nhận diện vấn đề.
 - Nhận diện các đối tượng cần thiết.
 - Nhận diện các message giữa các đối tượng.
 - Tạo các message giữa các đối tượng để giải quyết bài toán thực tế

Đặc trưng của thiết kế hướng đối tượng

- Không có vùng dữ liệu dùng chung.
 - Các đối tượng liên lạc với nhau bằng cách trao đổi thông báo chứ không phải các biến dùng chung.
- Các đối tượng là các thực thể độc lập, dễ thay đổi
- Các đối tượng có thể phân tán và có thể hành động tuần tự hoặc song song

Các ưu điểm, nhược điểm của TKHĐT

- Ưu điểm:
 - Dễ bảo trì vì các đối tượng là độc lập
 - Các đối tượng là các thành phần dùng lại được. Một thiết kế có thể dùng lại các đối tượng đã được thiết kế trong các bản thiết kế trước đó.
 - Đạt tính dễ hiểu trong thiết kế nhờ:
 - Các lớp trong hệ thống tương ứng với các thực thể ngoài đời thực.
 - Các tương tác điều khiển tương ứng với các hoạt động của đối tượng ngoài thế giới thực.

Các ưu điểm, nhược điểm của TKHĐT

- Nhược điểm:
 - Việc minh định các đối tượng trong hệ thống là khó khăn.
 - Cách nhìn tự nhiên là cách nhìn hướng chức năng, do vậy việc thích nghi với hướng đối tượng cho người mới bắt đầu thường khó khăn hơn.



Nội dung

- Thiết kế hướng đối tượng:
 - Cách tiếp cận hướng đối tượng
 - Đặc trưng của thiết kế hướng đối tượng
 - Các ưu nhược điểm của thiết kế hướng đối tượng
 - – Mẫu thiết kế - Design patterns
 - Một số nhiệm vụ thiết kế đề cập trong bài học:
 - Thiết kế dữ liệu
 - Thiết kế xử lý
 - Thiết kế giao diện



Mẫu thiết kế

- Một mẫu thiết kế là một mô tả, một cách tái sử dụng kiến thức trừu tượng về:
 - Một vấn đề
 - Và bản chất giải pháp của nó.
- Mẫu thiết kế (Design Pattern) chủ yếu được dựa theo các nguyên tắc:
 - Lập trình hướng đến các interface và tính trừu tượng (program to an interface not implementation)
 - Ưu tiên object composite hơn là thừa kế

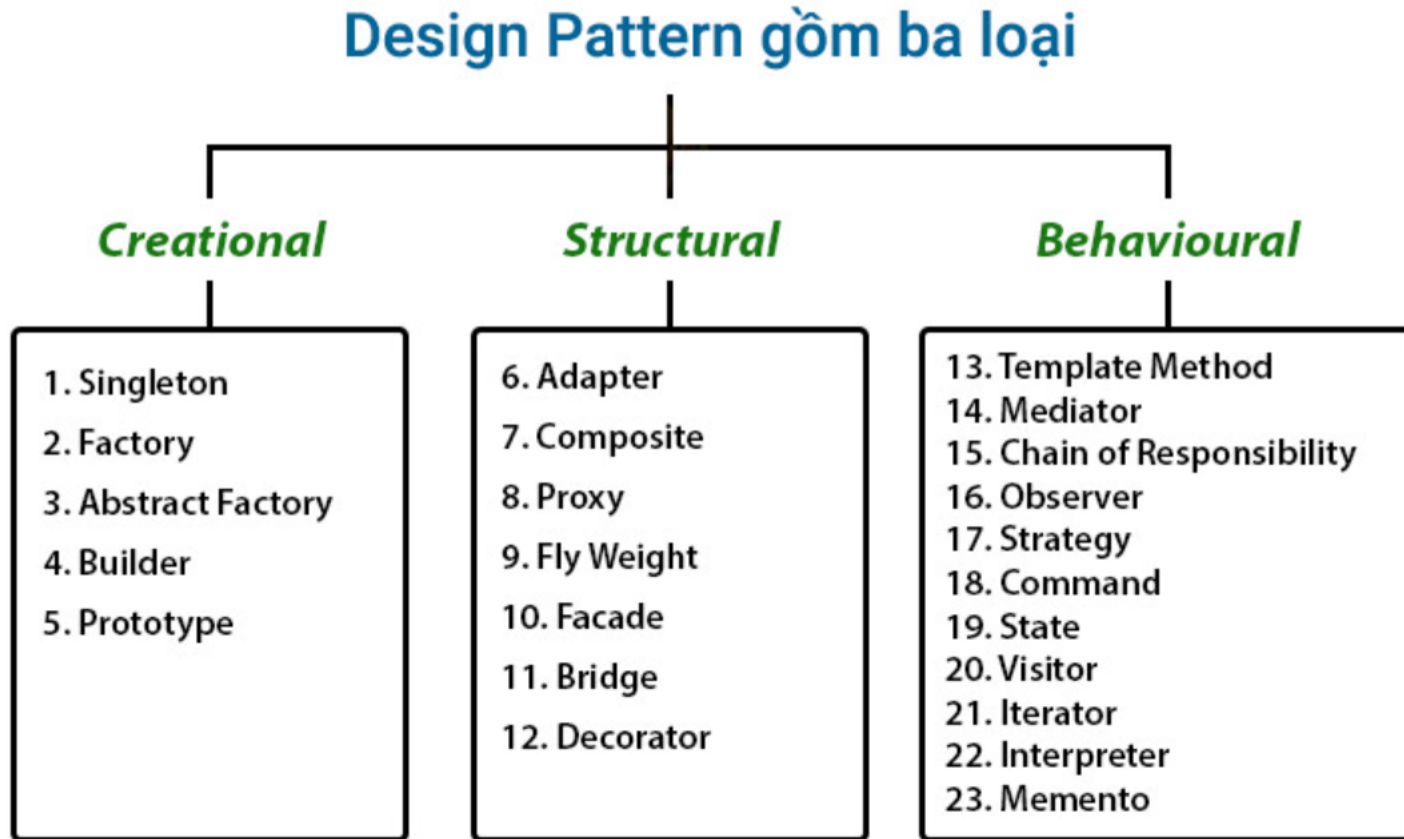


Phân loại mẫu thiết kế

- Design Patterns: Elements of Reusable Object Oriented Software, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, 1995 (Gang of Four)
- Creational Patterns
 - Abstract Factory, Builder, Factory Method, Prototype, Singleton
- Structural Patterns
 - Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
- Behavioral Patterns
 - Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor



Phân loại mẫu thiết kế





Các thành phần của một mẫu thiết kế

1. Tên mẫu thiết kế

- Là một tên gọi có ý nghĩa, ví dụ: Observer

2. Miêu tả vấn đề

3. Miêu tả giải pháp

- Không phải là một thiết kế cụ thể mà là một khuôn mẫu cho một giải pháp thiết kế có thể được khởi tạo cho nhiều cách khác nhau

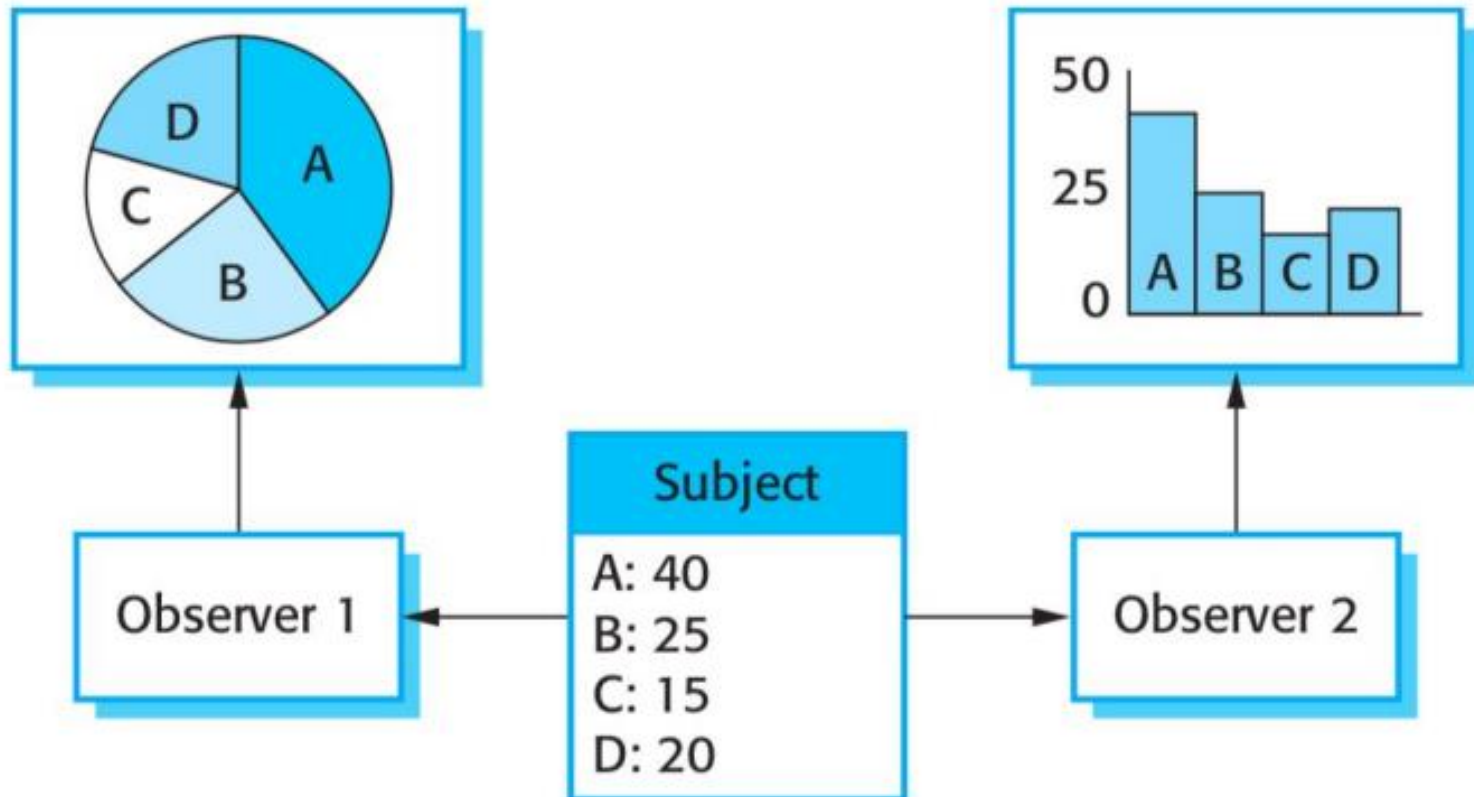
4. Các tác động

- Những kết quả và sự đánh đổi trong việc áp dụng mẫu.



Mẫu Observer

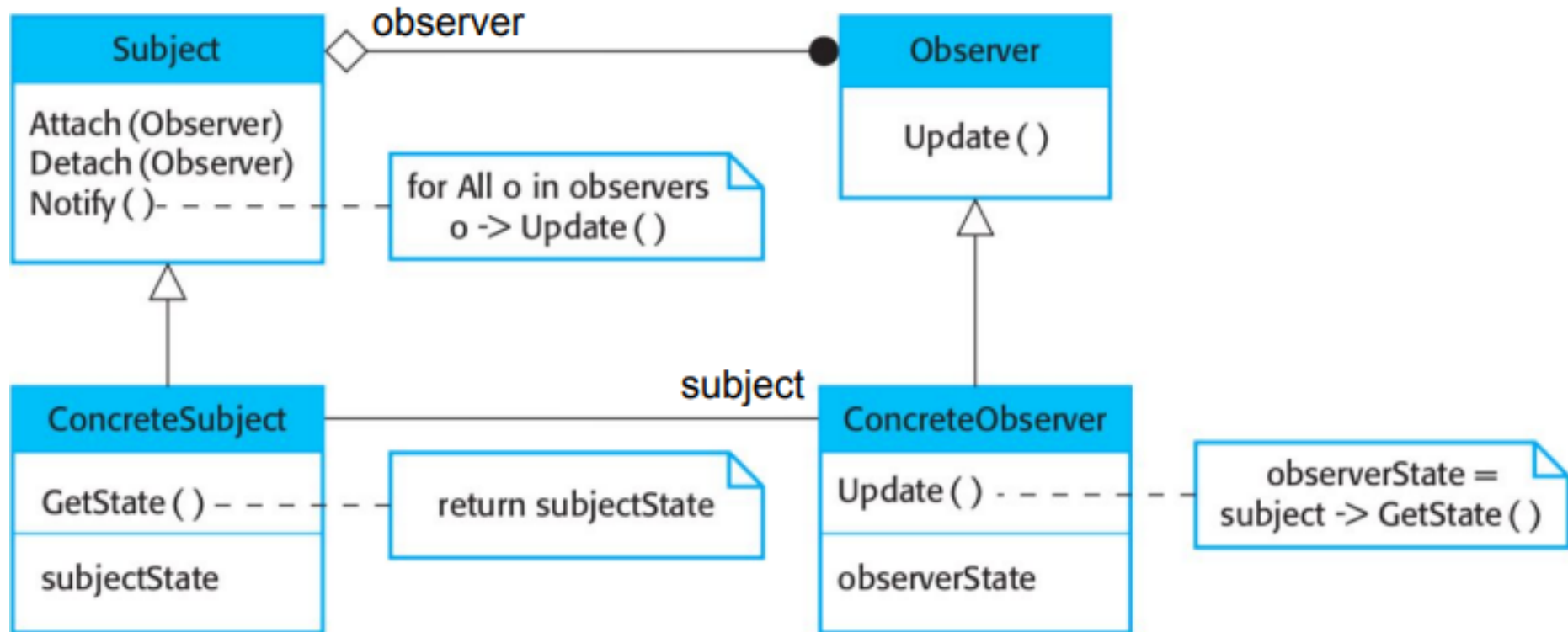
- Miêu tả vấn đề





Mẫu Observer

- Giải pháp





Nội dung

- Thiết kế hướng đối tượng:
 - Cách tiếp cận hướng đối tượng
 - Đặc trưng của thiết kế hướng đối tượng
 - Các ưu nhược điểm của thiết kế hướng đối tượng
 - Mẫu thiết kế - Design patterns
- - Một số nhiệm vụ thiết kế đề cập trong bài học:
 - Thiết kế dữ liệu
 - Thiết kế xử lý
 - Thiết kế giao diện



Thiết kế dữ liệu

- Mô hình hóa dữ liệu với mô hình dữ liệu quan hệ:
 - Có hai cách tiếp cận để mô hình hóa dữ liệu:
 - Cách 1: Tiếp cận Top-Down
Sử dụng biểu đồ thực thể liên kết ERD
 - Cách 2: Tiếp cận Bottom_up
Chuẩn hóa
- (Thiết kế dữ liệu được học trong môn học phần CSDL)



Thiết kế dữ liệu

- Chuẩn hóa dữ liệu từ hồ sơ dữ liệu sau:



CÔNG TY TNHH PHÁT TRIỂN THƯƠNG MẠI VÀ XUẤT NHẬP KHẨU HÀ LINH

Địa chỉ: Số nhà 10, ngõ 40, đường Kim Giang, Thanh Xuân, Hà Nội.

ĐT: 04.62918130 - 0912.961.001/0982.367.116 MST: 0106189683

Email: demhoiotosmart@gmail.com Website: demhoiotovn.com

HÓA ĐƠN BÁN HÀNG

Ngày.....tháng.....năm.....

Số:HL 00001

Khách hàng:.....Mã số thuế:.....

Địa chỉ:.....

Điện thoại:.....

STT	Mã hàng	Tên hàng hóa	ĐVT	S.lượng	Thực xuất	Đơn giá	Thành tiền
Thuế VAT 10%							
Tổng cộng							

Khách hàng
(ký & ghi rõ họ tên)

Người nhận hàng
(ký & ghi rõ họ tên)

Thủ kho
(ký & ghi rõ họ tên)

Nhân viên kế toán
(ký & ghi rõ họ tên)

Giám đốc
(ký & ghi rõ họ tên)



Thiết kế xử lý

- Khái niệm
- Một số chiến lược thiết kế xử lý
 - Thiết kế xử lý theo mô hình 3 tầng (3 layers)
 - MVC
 - MVP



Khái niệm thiết kế xử lý

- Với thể giới thực: Là **ng nghiệp vụ** của nhà chuyên môn.
- Với mức thiết kế: Là những thể hiện nghiệp vụ lên máy tính thành những **hàm, thủ tục**.
- Với mức lập trình: Là những **lệnh của một ngôn ngữ lập trình**.
- Với mức sử dụng: Là những **chức năng hay nút điều khiển** mà NSD sẽ chọn để thực hiện công việc của mình.



Kiến trúc các thành phần của PM



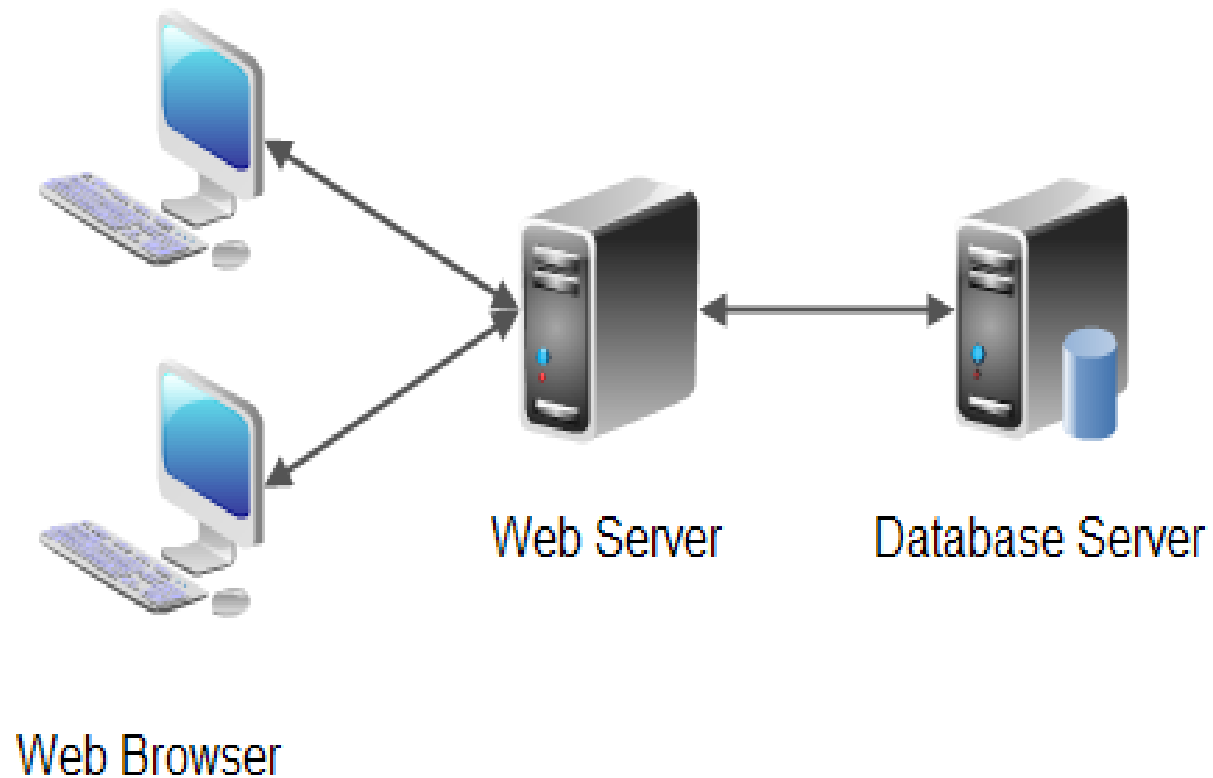


Kiến trúc ứng dụng dựa trên web

- Single-Tier : Tất cả các phân tầng (layer) của ứng dụng đều được tích hợp và cài đặt trên một máy tính.
- Two-Tier : Theo kiến trúc này cả 3 layer được phân bố cài đặt trên 2 tier đó là client và server.
- Three-Tier : Theo kiến trúc này 3 tầng (layer) sẽ được cài đặt tại các máy tính khác nhau.
- N-Tier



Mô hình 3 tier



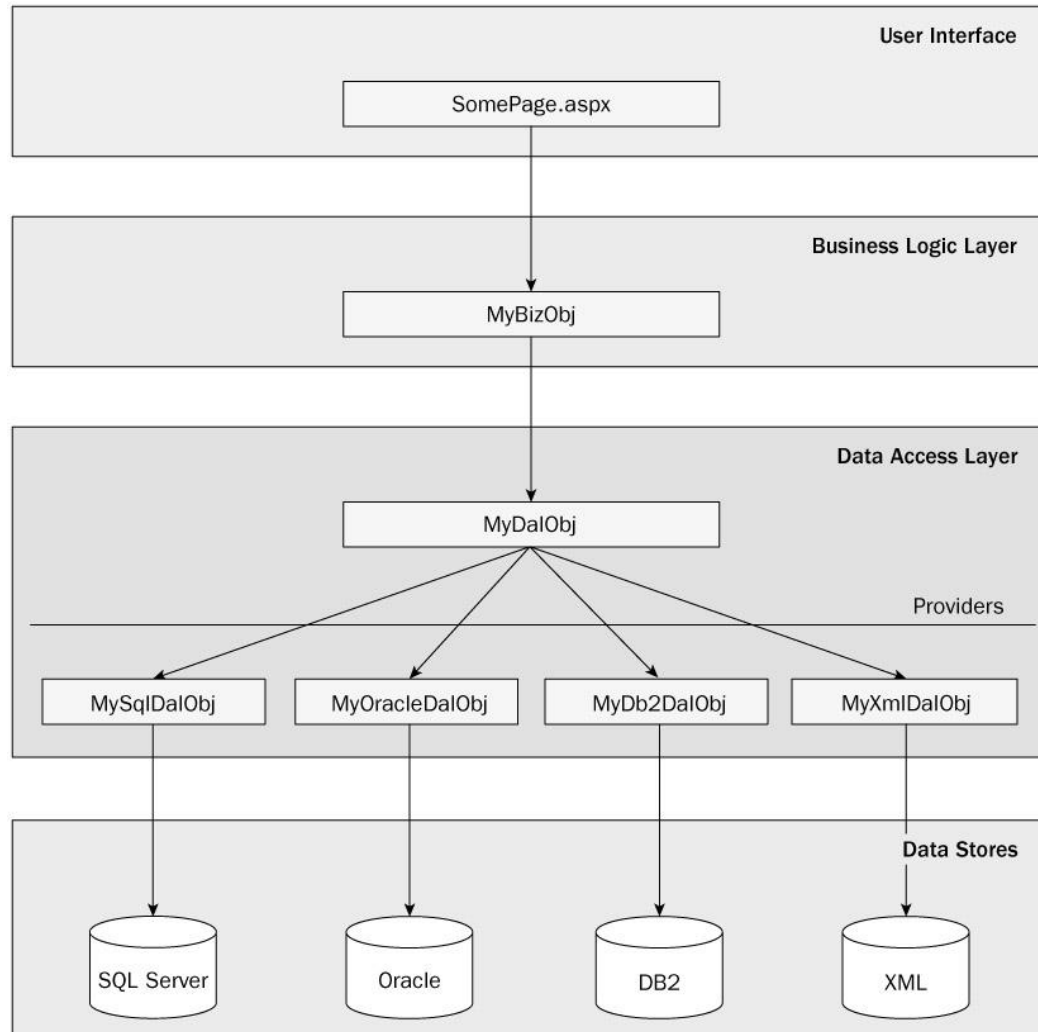


Chiến lược thiết kế xử lý

- Chia để trị
 - Đơn giản
 - Kế thừa
 - Dễ bảo trì, sửa chữa (Nếu có thay đổi, sai sót xảy ra thì số hàm bị ảnh hưởng ít)

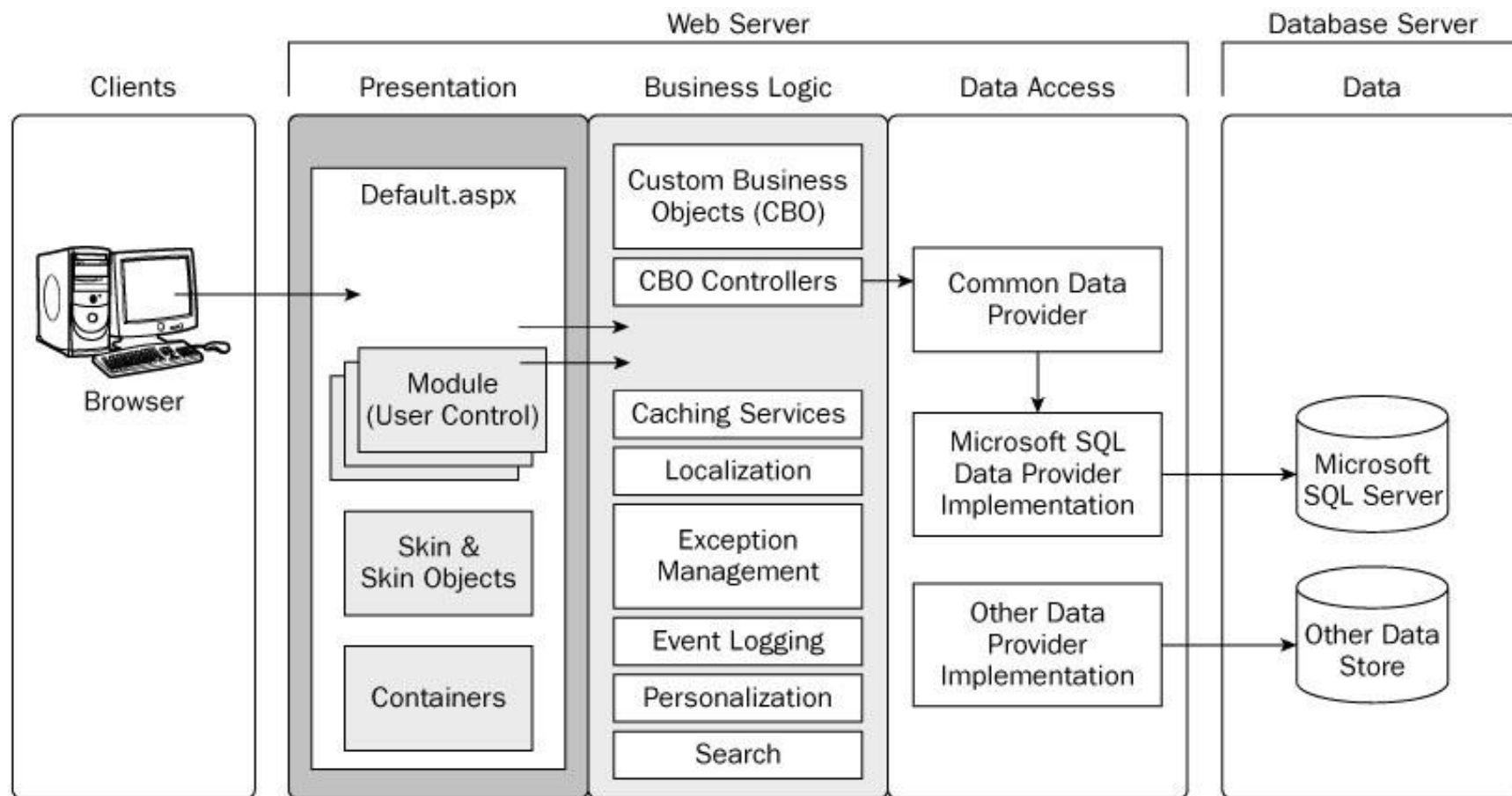


Thiết kế xử lý theo mô hình ba tầng





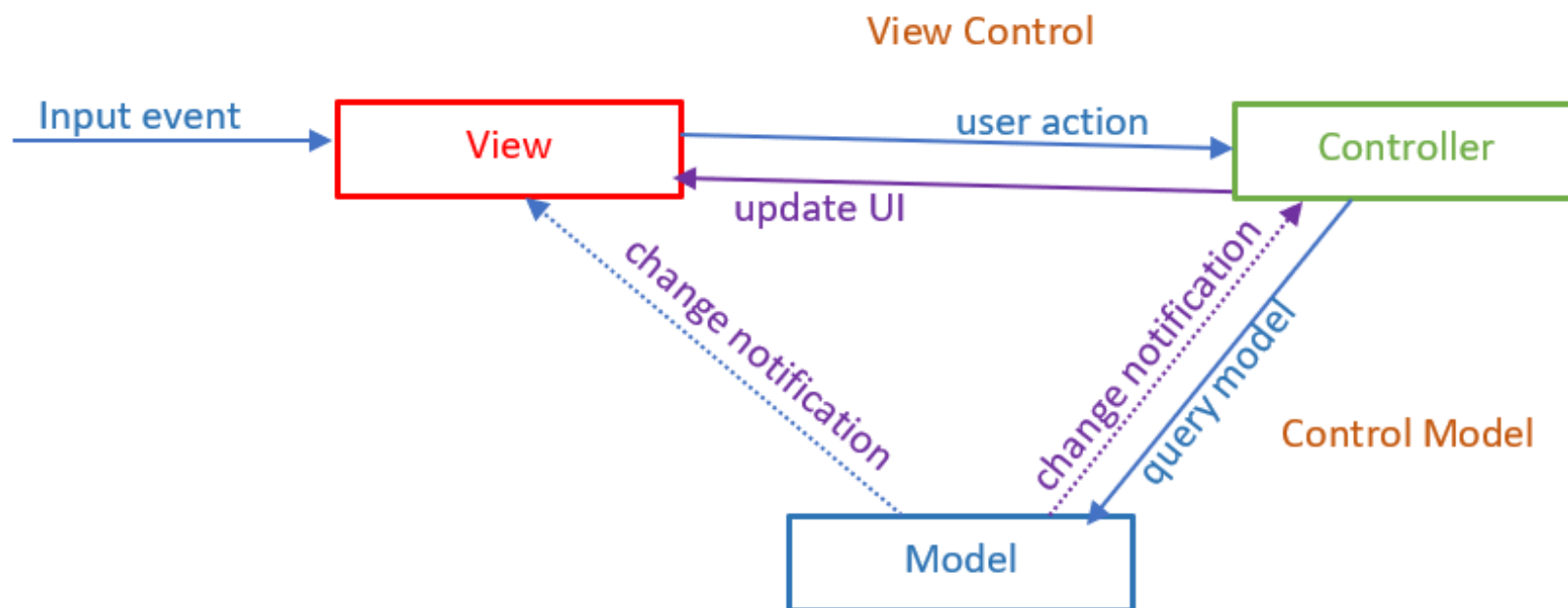
Thiết kế xử lý theo mô hình ba tầng





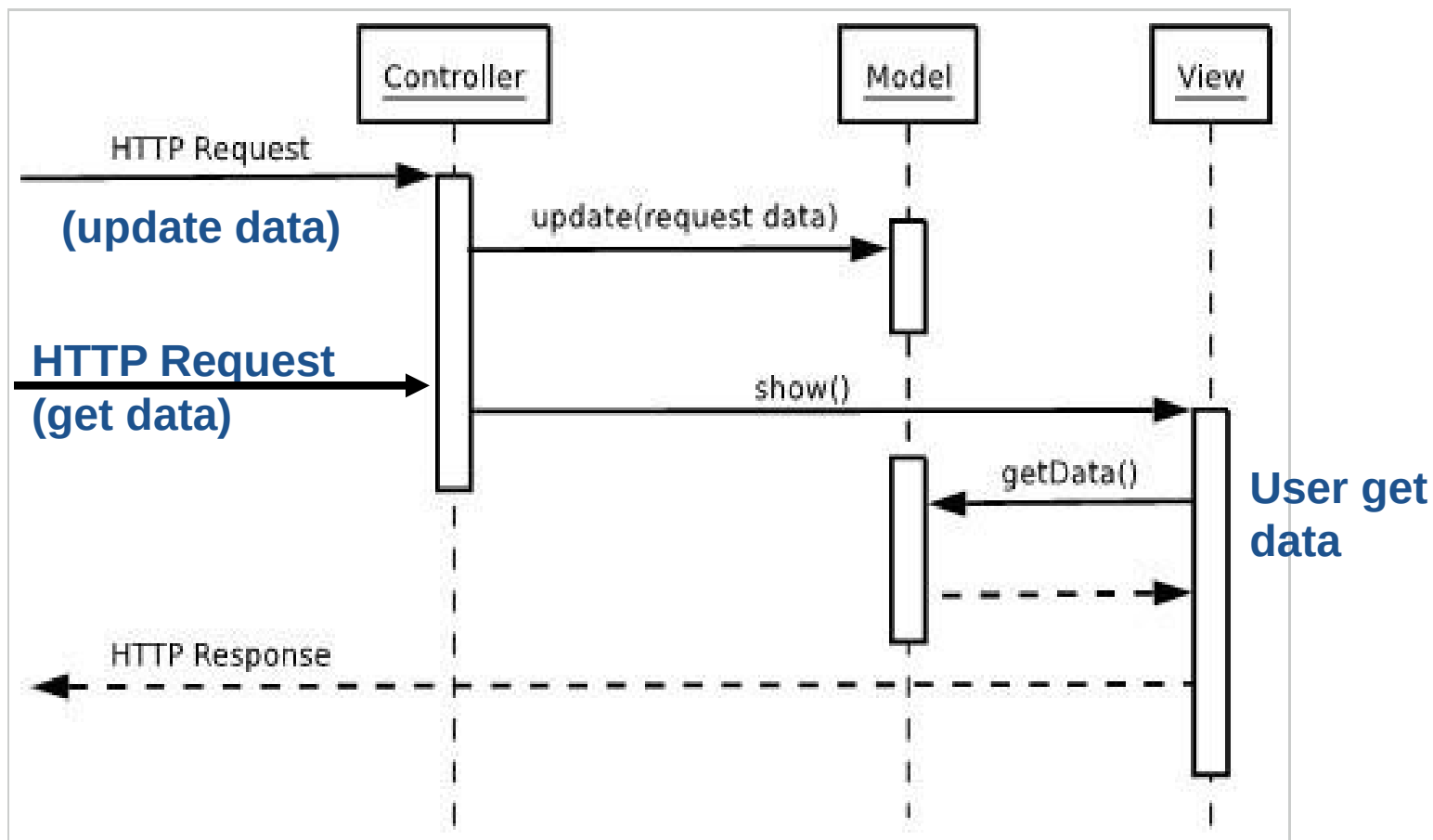
Mô hình kiến trúc phần mềm

- MVC





Biểu đồ tuần tự mô tả MVC





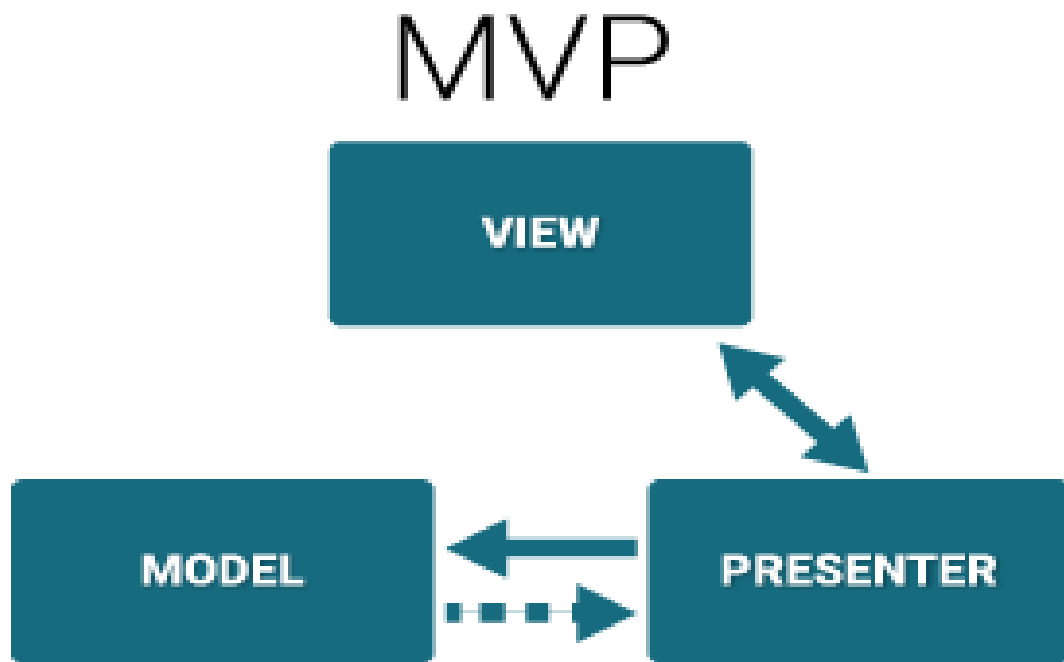
Các thành phần cơ bản của ứng dụng Asp.Net Core MVC

- Routing
- Model binding
- Model validation
- Dependency injection
- Filters
- Areas
- Web APIs
- Testability
- Razor view engine
- Strongly typed views
- Tag Helpers
- View Components



Thiết kế xử lý theo mô hình MVP

- Model – View – Presenter (Xu hướng trong lập trình di động – Android và GWT)

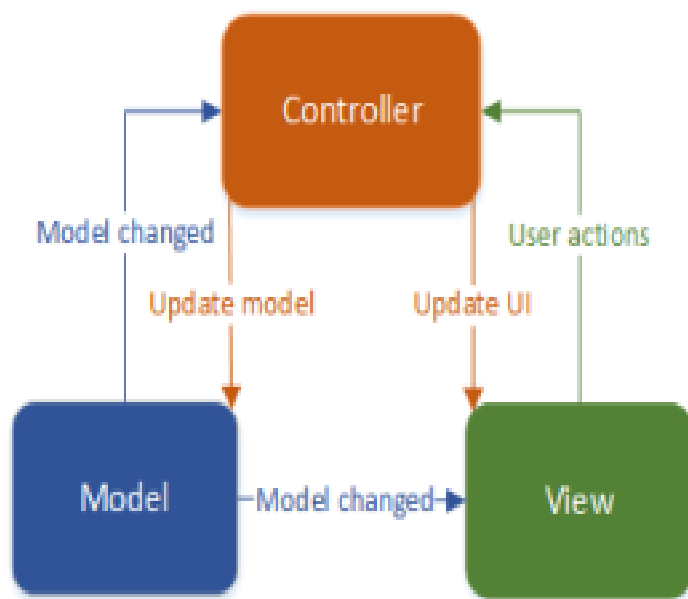




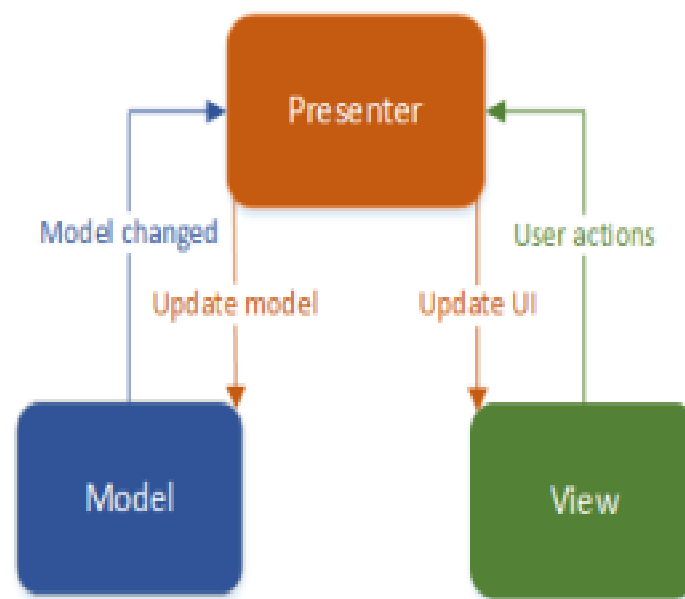
So sánh MVP và MVC

- Hai mô hình xử lý

MVC



MVP





Nội dung

- Thiết kế hướng đối tượng:
 - Cách tiếp cận hướng đối tượng
 - Đặc trưng của thiết kế hướng đối tượng
 - Các ưu nhược điểm của thiết kế hướng đối tượng
 - Mẫu thiết kế - Design patterns
 - Một số nhiệm vụ thiết kế đề cập trong bài học:
 - Thiết kế dữ liệu
 - Thiết kế xử lý
 - Thiết kế giao diện



Thiết kế giao diện

- Các vấn đề khi thiết kế
- Quy trình thiết kế UI
- Một số framework hỗ trợ UI cho các ứng dụng
 - Một số framework hỗ trợ UI cho phát triển web app
 - Một số framework hỗ trợ UI cho phát triển mobile web app
 - Một số framework hỗ trợ UI cho phát triển hybrid mobile app
- Autocomplete



Nhân tố con người trong thiết kế giao diện

- Trí nhớ ngắn hạn có hạn
 - Người ta có thể nhớ nhanh khoảng 7 thông tin.
 - Nếu bạn đưa ra nhiều hơn như vậy, người ta sẽ dễ nhầm lẫn.
- Nhầm lẫn là chuyện thường
 - Khi người dùng nhầm lẫn và hệ thống chạy sai, những cảnh báo và thông báo không đúng mức có thể làm người dùng căng thẳng hơn và càng dễ nhầm lẫn hơn.
- Mỗi người có năng lực cao thấp khác nhau
 - Người thiết kế không nên chỉ thiết kế cho năng lực của chính mình.
- Mỗi người hợp với những kiểu tương tác khác nhau
 - Người này thích hình ảnh, người khác thích chữ.



Các nguyên tắc chính thiết kế UI

- Thiết kế UI phải xét đến nhu cầu, kinh nghiệm và năng lực của người dùng hệ thống.
- Người thiết kế cần
 - nhận thức được các hạn chế về vật lý và tâm lý của con người
 - Ví dụ giới hạn của trí nhớ ngắn hạn
 - nhận thức rằng con người ai cũng có thể nhầm lẫn.
- Các nguyên tắc thiết kế UI đóng vai trò nền tảng cho các thiết kế giao diện
 - Tuy rằng không phải tất cả các nguyên tắc đều áp dụng được cho tất cả các thiết kế.



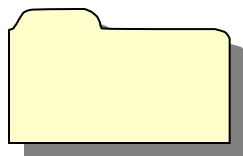
Các nguyên tắc thiết kế UI

Nguyên tắc	Mô tả
Quen thuộc với người dùng	Giao diện nên dùng các thuật ngữ và khái niệm rút ra từ kinh nghiệm của những người sẽ dùng hệ thống nhiều nhất.
Nhất quán	giao diện cần nhất quán sao cho các thao tác gần giống nhau có thể được kích hoạt theo cùng kiểu.
ngạc nhiên tối thiểu	Người dùng không bao giờ bị bất ngờ về hành vi của hệ thống.
khôi phục được	Giao diện nên có các cơ chế cho phép người dùng khôi phục lại tình trạng hoạt động bình thường sau khi gặp lỗi.
hướng dẫn người dùng	Giao diện nên có phản hồi có nghĩa khi xảy ra lỗi và cung cấp các tiện ích trợ giúp theo ngữ cảnh.
người dùng đa dạng	Giao diện nên cung cấp các tiện ích tương tác thích hợp cho các loại người dùng hệ thống khác nhau.



Các vấn đề khi thiết kế UI

- Hai vấn đề cần xem xét
 - Người dùng cung cấp thông tin cho hệ thống bằng cách nào?
 - Hệ thống nên trình bày thông tin (output) cho người dùng như thế nào?





Các kiểu tương tác vào dữ liệu

- Thao tác trực tiếp – Direct manipulation
- Chọn lựa bằng menu – Menu selection
- Điền form – Form fill-in
- Dòng lệnh – Command language
- Ngôn ngữ tự nhiên – a Natural language



WolframAlpha™ computational knowledge engine

Who is Vo Nguyen Giap?

Input interpretation: *Mathematica form*
Võ Nguyên Giáp (military)

Basic information:

full name	Võ Nguyên Giáp
date of birth	August 25, 1911 (age: 98 years)
place of birth	Vietnam

Timeline:

Võ Nguyên Giáp

1900 1925 1950 1975 2000

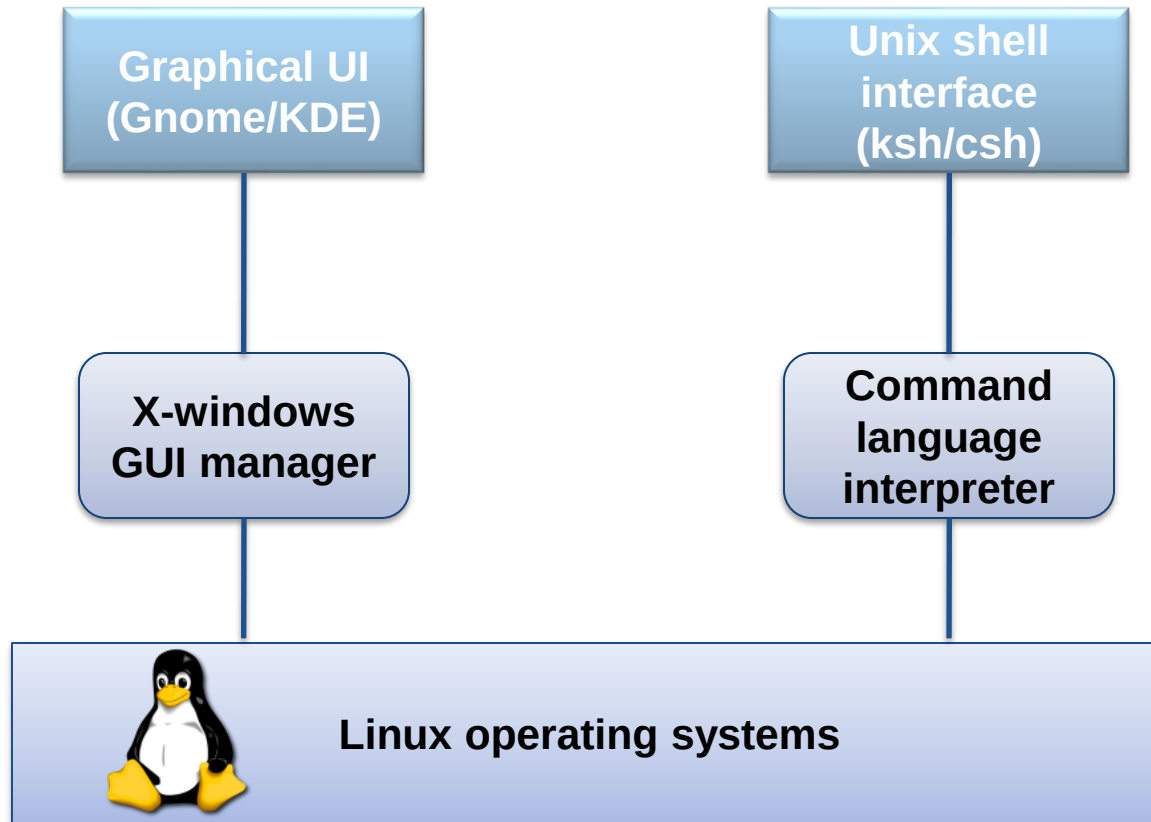


Các kiểu tương tác vào dữ liệu

Kiểu tương tác	Ưu điểm chính	Nhược điểm chính	Ví dụ ứng dụng
Direct manipulation	Tương tác nhanh và trực quan Dễ học	Có thể khó cài đặt. Chỉ thích hợp khi có ẩn dụ hình ảnh cho các tác vụ và đối tượng.	Trò chơi điện tử Các hệ thống dùng drag-n-drop
Menu selection	Tránh lỗi người dùng Không phải gõ nhiều	Chậm chạp đối với người dùng nhiều kinh nghiệm. Có thể phức tạp nếu có nhiều lựa chọn menu.	Đa số các hệ thống thông dụng
Form fill-in	Nhập dữ liệu đơn giản Dễ học Kiểm tra được	Tốn không gian màn hình. Gây rắc rối khi các lựa chọn của người dùng không khớp với các trường của form.	Khai thuế, xử lý nợ cá nhân
Command language	mạnh và linh động	Khó học xử lý lỗi kém	hệ điều hành
Natural language	Đáp ứng được người dùng không chuyên Dễ mở rộng	Cần gõ nhiều. Các hệ thống hiểu ngôn ngữ tự nhiên không đáng tin cậy.	Information retrieval systems



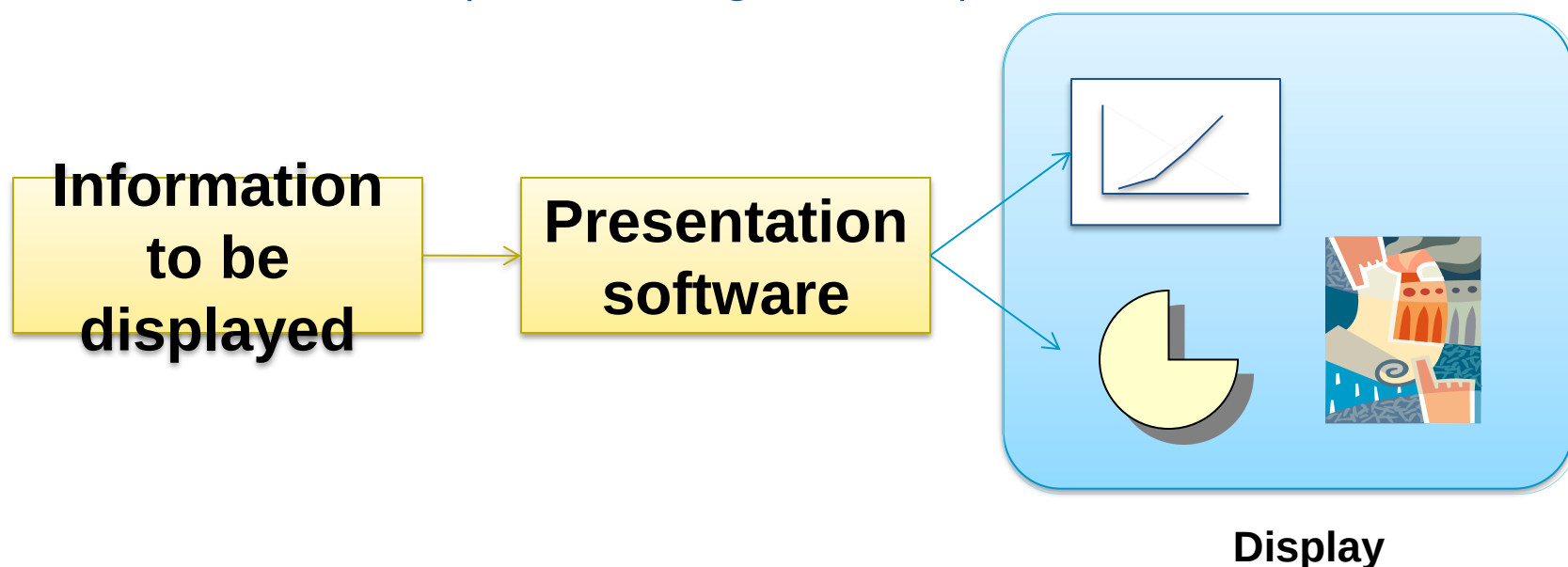
Multiple user interfaces





Biểu diễn thông tin ra (output)

- Trình bày thông tin hệ thống như thế nào cho người dùng.
- Thông tin có thể được trình bày trực tiếp (ví dụ text trong một trình soạn thảo) hoặc được biến đổi thành một dạng biểu diễn khác (ví dụ dạng đồ họa)





Biểu diễn thông tin ra (output)

- Thông tin tĩnh
 - Khởi tạo ở đầu session. Không thay đổi trong suốt session.
- Thông tin động
 - Thay đổi trong session và phải được thông báo với người dùng.



Các nhân tố khi hiển thị thông tin

- Người dùng quan tâm đến con số chính xác hay mối tương quan giữa các số liệu?
- Các giá trị của thông tin thay đổi nhanh chậm ra sao? Có phải lập tức thông báo khi có thay đổi?
- Người dùng có phải hành động để phản ứng với một thay đổi?



Các thông báo lỗi

- Thiết kế thông báo lỗi là việc rất quan trọng
 - Các thông báo lỗi kém có thể dẫn đến việc người dùng không chấp nhận sử dụng một hệ thống.
- Các thông báo nên lịch sự, ngắn gọn, nhất quán và mang tính xây dựng.
- Nên xét đến background và kinh nghiệm của người dùng như là nhân tố quyết định khi thiết kế các thông báo lỗi.



Các nhân tố khi thiết kế nội dung các thông điệp

Nhân tố	Miêu tả
Ngữ cảnh	Các thông điệp do hệ thống tạo ra cần phản ánh ngữ cảnh sử dụng hiện tại bất cứ khi nào có thể. Hệ thống cần cố gắng nhận biết được người dùng đang làm gì và nên tạo các thông điệp có liên quan đến hoạt động hiện tại của họ.
Kinh nghiệm	Khi người dùng quen dần với một hệ thống, họ sẽ trở nên khó chịu bởi các thông điệp dài 'có ý nghĩa'. Tuy nhiên, những người mới dùng lại thấy những thông báo ngắn gọn khó hiểu. Bạn nên cung cấp cả hai loại thông điệp và để cho người dùng kiểm soát mức độ xúc tích của các thông báo.
Trình độ kĩ năng	Các thông điệp nên được điều chỉnh theo trình độ kĩ năng cũng như kinh nghiệm của người dùng. Thông điệp dành cho các loại người dùng khác nhau có thể được diễn đạt theo các cách khác nhau tùy theo bộ thuật ngữ quen thuộc với người đọc.
Phong cách	Các thông điệp nên mang tính tích cực hơn là tiêu cực, nên dùng thể chủ động hơn là bị động, không bao giờ nên có giọng xúc phạm hoặc cố pha trò.
Văn hóa	Nếu có thể, người thiết kế nên là người quen thuộc với văn hóa của đất nước nơi hệ thống sẽ được bán. Có những khác biệt văn hóa rõ rệt giữa châu Âu, châu Á và châu Mỹ. Một thông điệp phù hợp với nền văn hóa này có thể không chấp nhận được ở một nền văn hóa khác.



Lỗi người dùng

- Giả sử một y tá đang cần tìm hồ sơ của một bệnh nhân nhưng lại gõ sai tên của bệnh nhân này.

Mời nhập tên của bệnh nhân vào hộp bên dưới rồi nhấn OK

Tên bệnh nhân

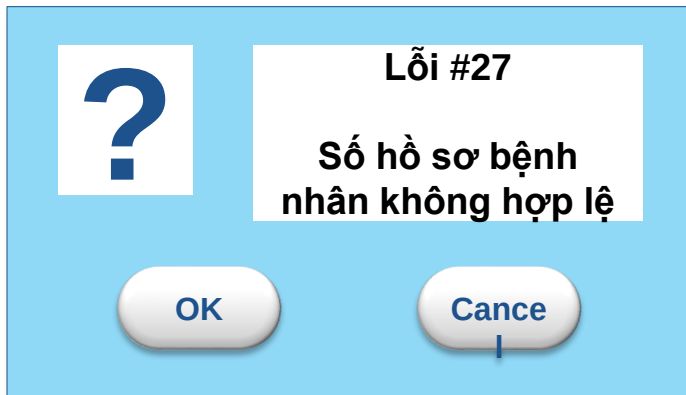
MacDonald, R.

OK Cancel

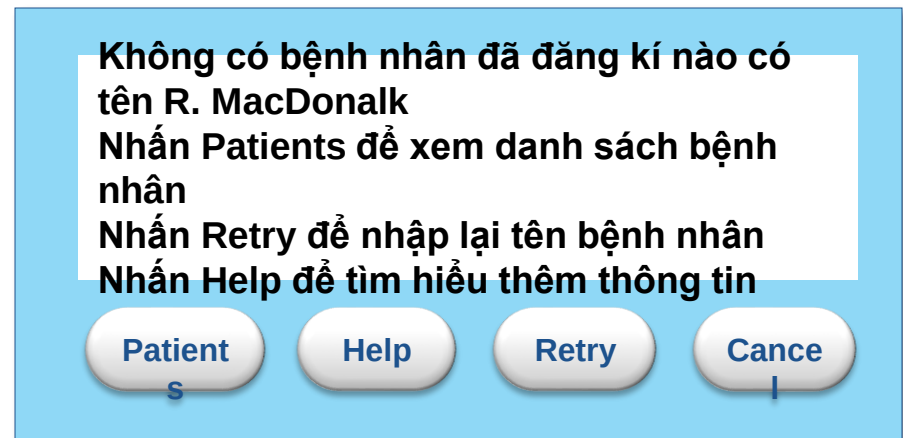


Thiết kế thông điệp tốt và tồi

Thông báo lỗi kiểu hệ thống



Thông báo lỗi kiểu người dùng



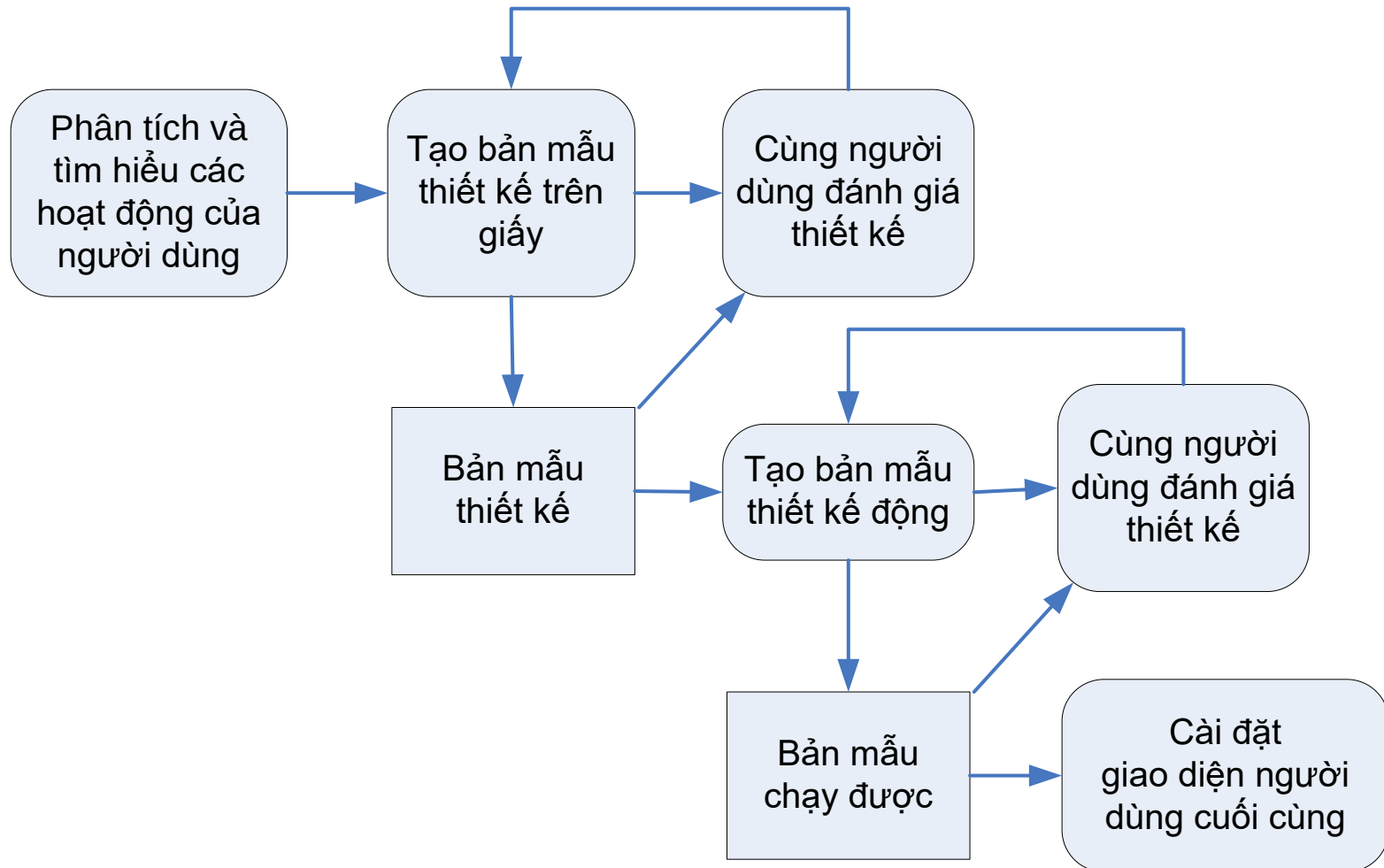


Quy trình thiết kế UI

- Thiết kế UI là một quy trình lặp đi lặp lại với sự liên lạc chặt chẽ giữa người dùng và người thiết kế.
- Ba hoạt động chính trong quy trình:
 - **User analysis:** Tìm hiểu người dùng sẽ làm gì với hệ thống;
 - **System prototyping:** phát triển một loạt các bản mẫu để thử nghiệm;
 - **Interface evaluation:** thử nghiệm các bản mẫu cùng với người dùng.



Quy trình thiết kế





Phân tích người dùng

- Nếu bạn không hiểu người dùng muốn làm gì với một hệ thống, bạn khó có thể thiết kế một giao diện hiệu quả.
- Các phân tích người dùng phải được mô tả theo cách mà cả người dùng lẫn người thiết kế đều có thể hiểu được.
- Các kịch bản mà bạn dùng để miêu tả các tình huống sử dụng điển hình là một cách để mô tả các phân tích này.



Các kỹ thuật phân tích người dùng

- Phân tích tác vụ - Task analysis
 - Mô hình hóa các bước trong việc hoàn thành một tác vụ.
- Phỏng vấn và câu hỏi điều tra (questionnaire)
 - Hỏi người dùng về công việc của họ.
- Văn hóa
 - Quan sát người dùng khi họ đang làm việc.



Bản mẫu thiết kế-UI

- Mục tiêu của bản mẫu là cho phép người dùng có được trải nghiệm trực tiếp đối với giao diện.
- Nếu không có trải nghiệm trực tiếp, không thể đánh giá khả năng sử dụng một giao diện.
- Quy trình làm bản mẫu có thể có 2 bước:
 - Lúc đầu, có thể dùng bản mẫu trên giấy;
 - Sau đó tinh chỉnh thiết kế, và phát triển các bản mẫu tự động hóa với độ phức tạp ngày càng tăng.



Bản mẫu thiết kế-trên giấy

- Dùng các bản vẽ nháp về UI để duyệt qua các kịch bản.
- Dùng storyboard để trình bày một chuỗi tương tác với hệ thống.
- Bản mẫu giấy là cách thu thập phản ứng của người dùng đối với một đề xuất thiết kế.



Các kỹ thuật làm bản mẫu

- Script-driven prototyping
 - Viết một loạt các script và screen bằng các công cụ. Ví dụ như figma...
 - Khi người dùng tương tác, screen sẽ chuyển sang hiển thị màn hình tiếp theo.
- Visual programming
 - Dùng một ngôn ngữ chuyên dụng cho phát triển nhanh như Visual Basic.
- Internet-based prototyping
 - Dùng một trình duyệt web browser và các script kèm theo.



Các kỹ thuật làm bản mẫu

- Hướng dẫn sử dụng FIGMA



Thảo luận

